

Neural Networks

Neuron



$$z = wx + b$$

$$\hat{y} = f(z)$$

↑

Forward pass

Back propagation: $w' = w - \eta \times \frac{\partial \hat{y}}{\partial w}$ ↗ Learning rate

$$\frac{\partial \hat{y}}{\partial w} = \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w} = x f'(z)$$

$\downarrow$
 $f'(z)$

Work it out for $\frac{\partial \hat{y}}{\partial b} \longrightarrow$

$$\hat{y} = f(wx + b)$$

→ Activation Functions

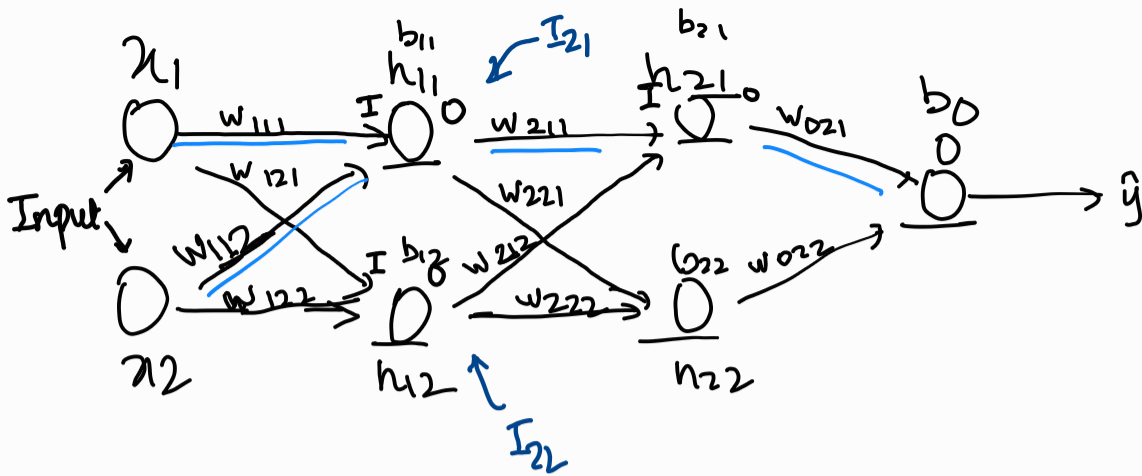
$\max(0, x)$ ReLU ~~+~~

$1/(1+e^{-x})$ Sigmoid ~~+~~

$(e^x - e^{-x})/(e^x + e^{-x})$ TanH ~~+~~

$\max(0.01x, x)$ LeakyRelu ~~+~~

Deep Neural Network



$$I_{11} = w_{111}x_1 + w_{121}x_2 + b_{11}$$

$$I_{12} = w_{121}x_1 + w_{122}x_2 + b_{12}$$

$$O_{11} = f(I_{11})$$

$$O_{12} = f(I_{12})$$

$$I_{21} = w_{211}O_{11} + w_{212}O_{12} + b_{21}$$

$$I_{22} = w_{221}O_{11} + w_{222}O_{12} + b_{22}$$

$$O_{21} = f(I_{21})$$

$$O_{22} = f(I_{22})$$

$$X = [x_1 \ x_2]$$

$$W_1 = \begin{bmatrix} w_{11} & w_{12} \\ 1 & 1 \end{bmatrix}$$

$$Z = W_1 X^T + B_1$$

$$B_1 = [b_{11} \ b_{12}]$$

$$O_1 = f(Z) \rightarrow [f(z_1) \ f(z_2)]$$

$$Z = W_2 O_1^T + B_2$$

$$\hat{y} = \text{softmax}(Z)$$

$$\hat{y} = \text{softmax}(O) \rightarrow \text{Probability}$$

$$L = -\sum_{i=1}^n y_i \log(\hat{y}_i) \rightarrow \text{Cross entropy loss}$$

Intuition $\rightarrow$ $\underbrace{[1, 0, 0]}_{\text{Truth}} \quad \underbrace{[0.75, 0.15, 0.1]}_{\text{Pred.}}$

$$L = -\log_2 0.75 \approx 0.4150374$$

Back-propagation

To find the losses $\rightarrow$

$$\nabla_y L = y_i \nabla \log(\hat{y}_i)$$

$$\nabla \hat{y} = f(\nabla O)$$

$$\nabla O = \nabla W_1 \odot x^T + b$$

$$W = W - \eta \nabla W$$

Activation Functions & Optimizers

ReLU, GeLU, SiLU, tanh, Sigmoid

↓ ↓
[Vanishing & Exploding]

Activation Func: $\rightarrow$ Pre-req,
 $\rightarrow$ Differentiable.

$$\text{ReLU} = \max(0, x)$$

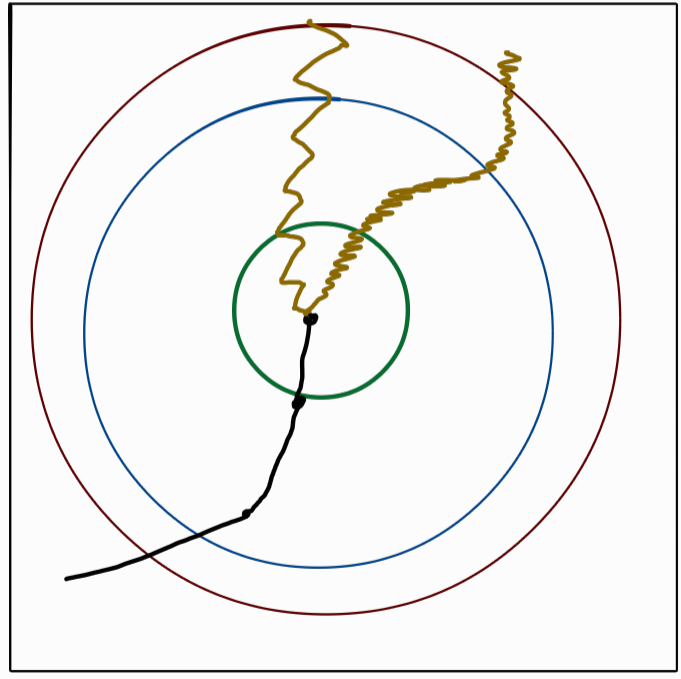


$w \leftarrow \eta \times \text{Gradient}$



minibatch $\rightarrow$ SGD $\rightarrow$ LO

Momentum-based GD
Adam
AdaGrad
 $\downarrow$ scheduling



Kernels

MNIST $\rightarrow$ 28x28 image
 $\downarrow$
Neural network (flattening)

Images $\nrightarrow$ No neural networks *only*

$\rightarrow$ Too much to compute $\checkmark$ parameters
 $\rightarrow$ Spatial information loss. $\checkmark$

Parameters $\rightarrow$ weight, biases

Kernels:

Laplacian $\rightarrow$ $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$

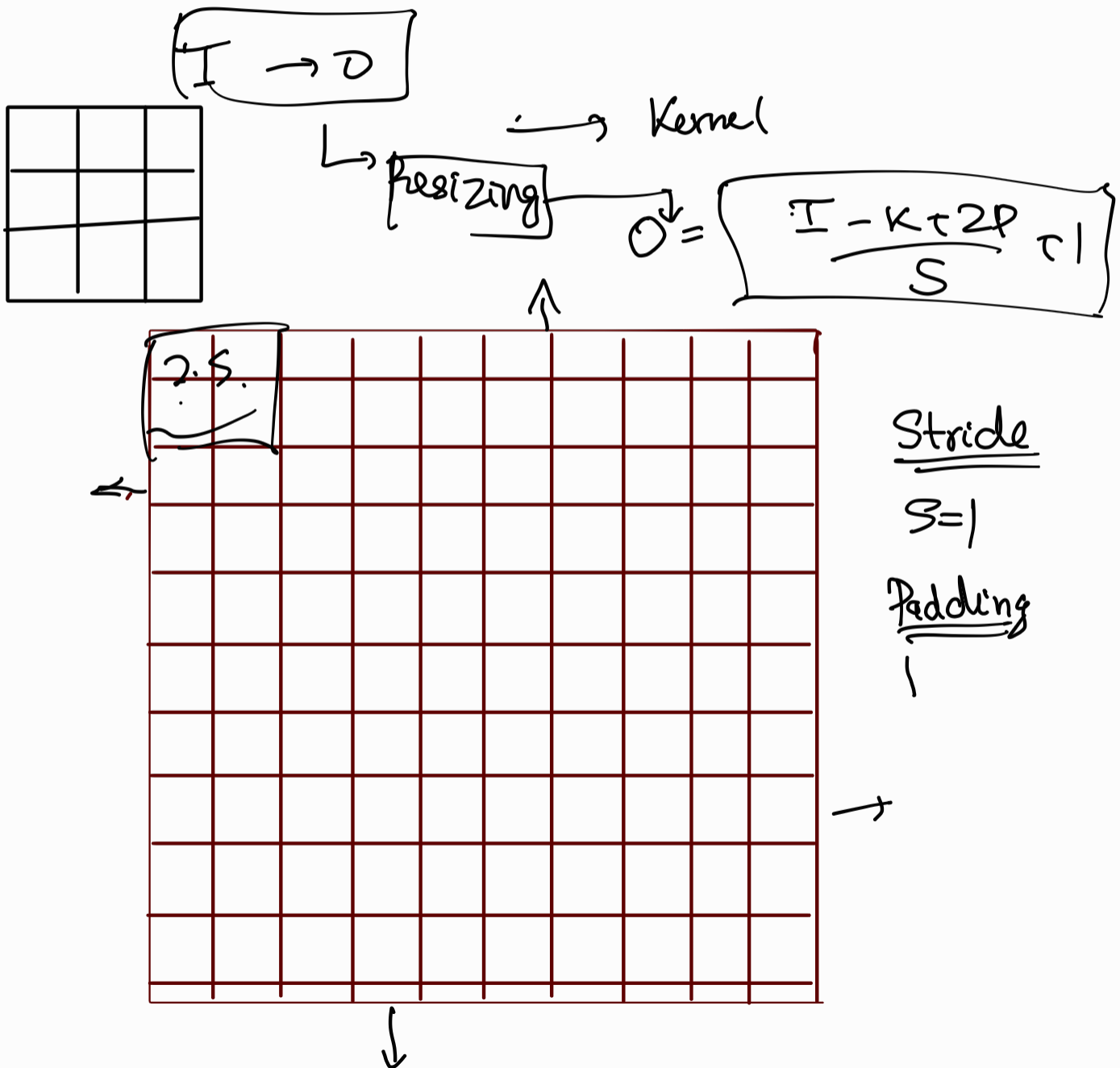
Sobel-x $\rightarrow$ $\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

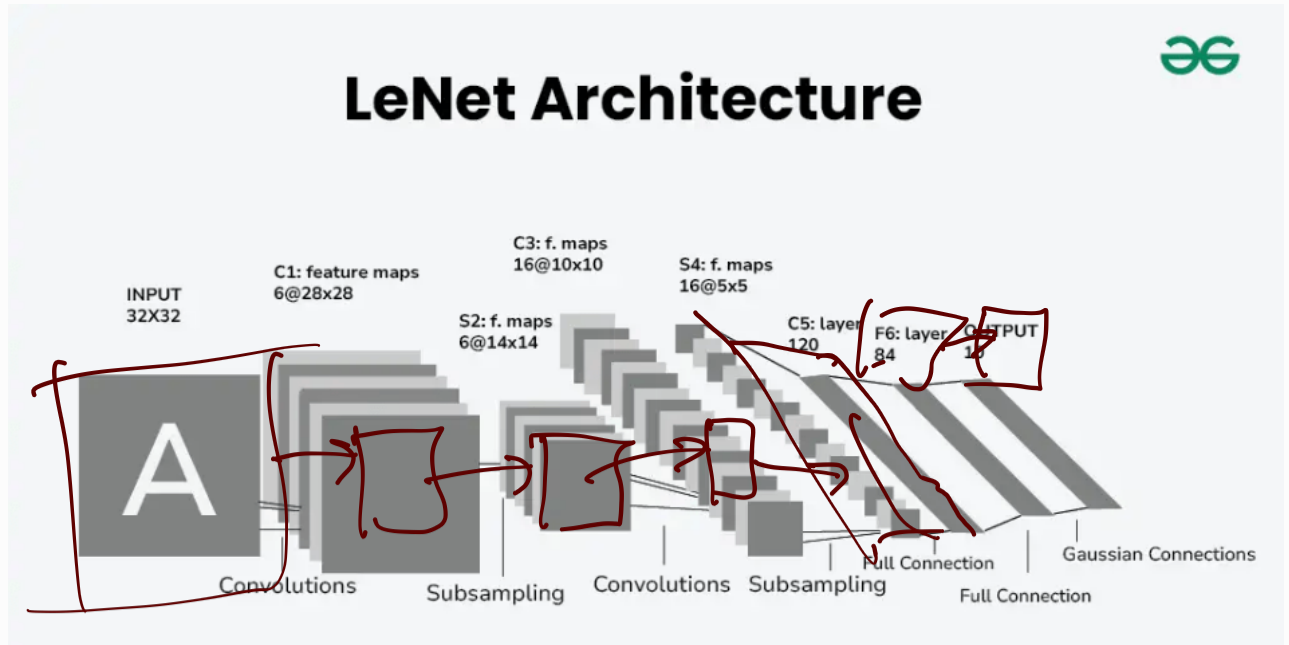
$$f \star g = \int_{-\infty}^{\infty} f(t-\tau) g(\tau) d\tau$$

$I \rightarrow (W, H, \text{Channels})$

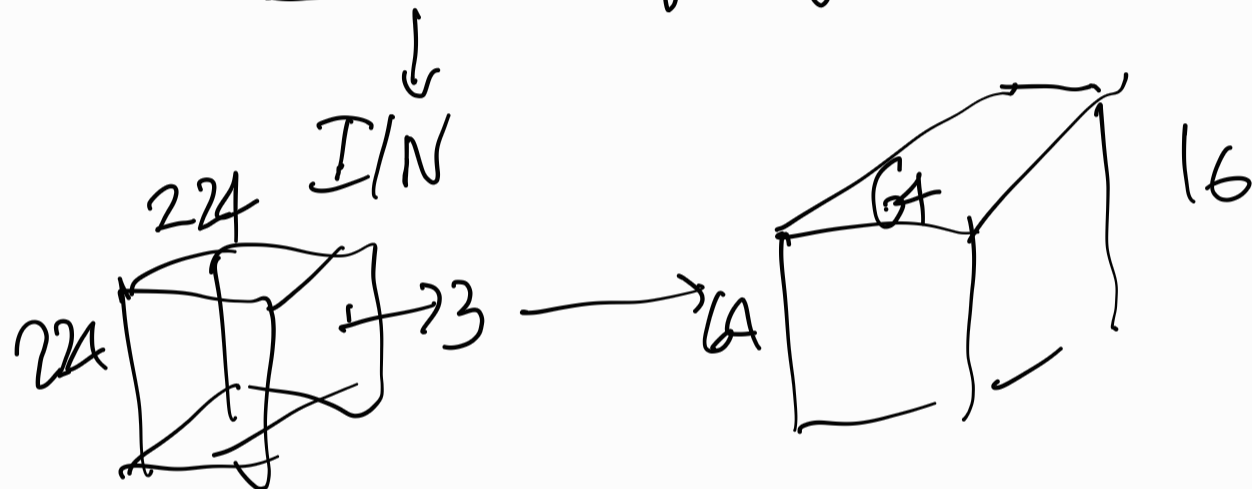
$I * K \rightarrow \text{Next layers}$



LeNet



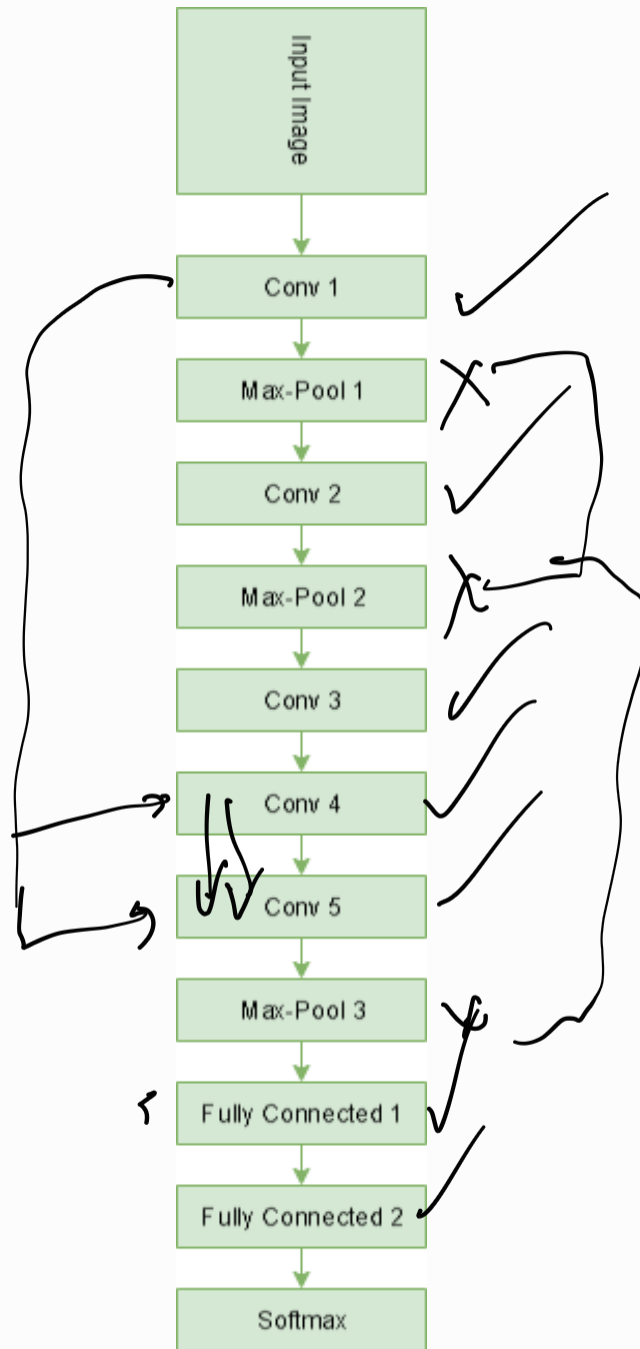
Sample: MaxPooling(CN), ArgPooling(CN)



7x7x512 → Flatten →

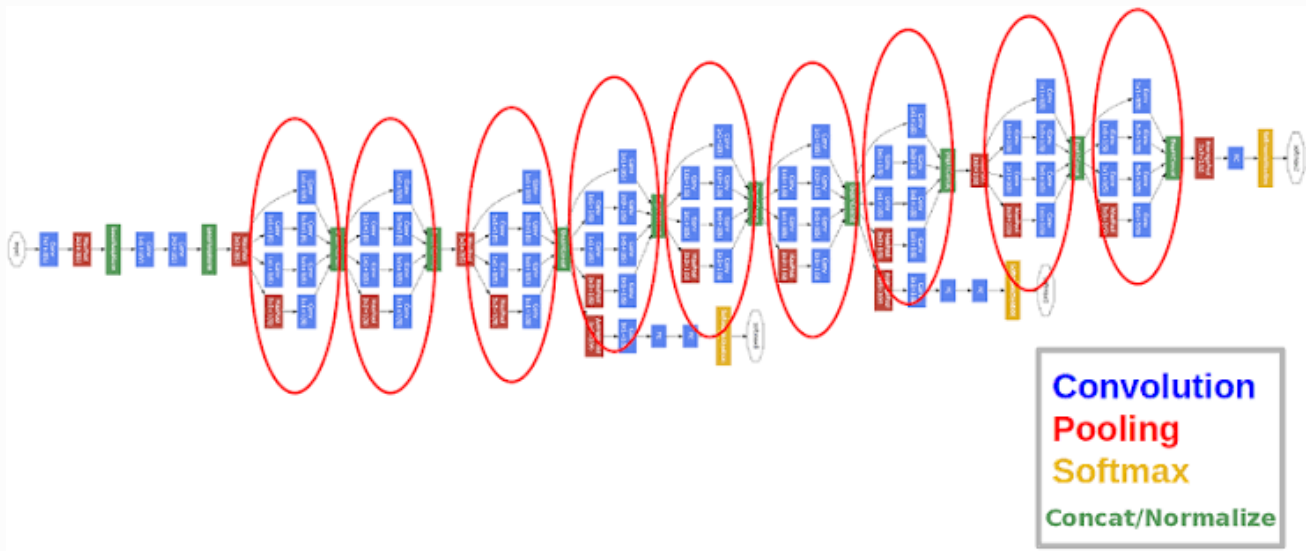
ImageNet → AlexNet (2011, ImageNet)
1000 images

AlexNet



7 layers

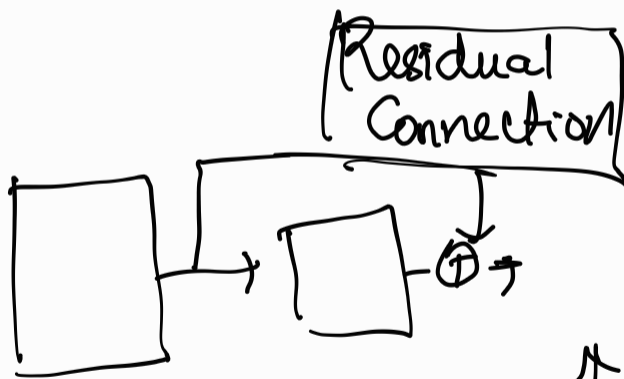
Vanishing gradients



2014 - GoogLeNet

2015 → ResNet

CNNs End Here (for us!)



Resnet-34, Resnet-50 ↑